

MedGAN

Model 3 | Subset ZINC15-III | Reinforcement Learning | 100k quinolines

```
# import packages

import numpy as np
import os
import pandas as pd
import datetime
import gc
import io
import json
import csv
import matplotlib.lines as mlines
import matplotlib.pyplot as plt
import networkx as nx
from rdkit.Chem import rdmolops
from rdkit.Chem.rdmolops import AddHs
from rdkit.Chem.rdmolops import GetMolFragments
from rdkit import Chem, RDLogger
from rdkit import DataStructs
from rdkit.Chem import AllChem, Draw, Descriptors
from rdkit.Chem import AtomValenceException
from rdkit.Chem import Descriptors
from rdkit.Chem import MolFromSmiles
from rdkit.Chem import rdMolDescriptors
from rdkit.Chem import rdmolfiles
from rdkit.Chem.Draw import IPythonConsole, MolsToGridImage
from rdkit.Chem.Fingerprints import FingerprintMols
from rdkit.Chem.Scaffolds import MurckoScaffold
from rdkit.DataStructs.cDataStructs import TanimotoSimilarity
import rdkit.RDLogger as rdl
from tensorflow import keras
from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.callbacks import TensorBoard
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.utils import plot_model
import tensorflow as tf
from keras import initializers
from keras.layers import BatchNormalization
from scipy.spatial.distance import pdist, squareform
import concurrent.futures
from tqdm import tqdm
import base64
from IPython.display import display
```

```

from IPython.display import Image
from multiprocessing import Pool, cpu_count
from PIL import Image
import gzip
import pickle
import psutil
import pygraphviz as pgv
import time
import shutil
RDLogger.DisableLog("rdApp.*")
tf.get_logger().setLevel('ERROR')
logger = rdl.logger()
logger.setLevel(rdl.ERROR)
logger.setLevel(rdl.CRITICAL)

2025-03-04 16:42:07.863332: I
tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow
binary is optimized to use available CPU instructions in performance-
critical operations.
To enable the following instructions: AVX2 FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
2025-03-04 16:42:09.445174: W
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning:
Could not find TensorRT

```

Data Preprocessing: Filtering and Sampling of Quinoline Molecules

The following section is dedicated to loading and processing quinoline molecule data from the ZINC15 dataset. Initially, the script checks whether a pre-filtered dataset (non_duplicate_filtered_quinolines_zinc15_50atoms.csv) exists. If it doesn't, the original dataset (non_duplicate_quinolines_zinc15.csv) is loaded and molecules are filtered based on specific criteria:

Only molecules containing the atoms - Carbon (C), Nitrogen (N), Oxygen (O), Hydrogen (H), Fluorine (F), Sulfur (S), and Chlorine (Cl) are retained.

Molecules having up to a maximum of 50 atoms, including hydrogen, are considered.

Post filtering, duplicates are removed, and the processed data is saved for future use. If the filtered dataset already exists, it is directly loaded.

To ensure consistent results and manageable data sizes, a subsample of 100,000 molecules is randomly chosen from this dataset. This subsample is then saved for subsequent analyses. Lastly, as a quick verification, one molecule is printed to visualize its structure and compute its count of heavy atoms.

```

# Load filtered csv or save a new one
csv_path =

```

```

"/home/ec2-user/MedGAN/datasets/zinc15_quinolines_data/non_duplicate_f
iltered_quinolines_zinc15_50atoms.csv"
filtered_csv_path =
"/home/ec2-user/MedGAN/datasets/zinc15_quinolines_data/non_duplicate_f
iltered_quinolines_zinc15_50atoms.csv"

allowed_atoms = {'C', 'N', 'O', 'H', 'F', 'S', 'Cl'}
max_atoms = 50

# Functions

def has_allowed_atoms(mol, allowed_atoms):
    for atom in mol.GetAtoms():
        if atom.GetSymbol() not in allowed_atoms:
            return False
    return True

def total_atoms(mol):
    return sum(atom.GetTotalNumHs() + 1 for atom in mol.GetAtoms())

def is_valid_smiles(smiles):
    molecule = Chem.MolFromSmiles(smiles)
    molecule = Chem.AddHs(molecule)
    if (molecule is not None
        and '*' not in smiles
        and has_allowed_atoms(molecule, allowed_atoms)
        and total_atoms(molecule) <= max_atoms):
        return smiles
    return None

def get_cpu_usage():
    return psutil.cpu_percent()

# If filtered CSV doesn't exist, read the original CSV, apply the
filter, and save the filtered data
if not os.path.isfile(filtered_csv_path):
    # Read CSV file
    data = pd.read_csv(csv_path, usecols=lambda col: col != 'index',
header=None)

    valid_smiles = []
    for smiles in tqdm(data[0]):
        try:
            result = is_valid_smiles(smiles)
            if result is not None:
                valid_smiles.append(result)
        except Exception as e:
            print("Error processing item:", e)

    # Remove duplicates

```

```

valid_smiles = list(set(valid_smiles))

# Create DataFrame with valid SMILES
data = pd.DataFrame(valid_smiles, columns=['smiles'])

# Save the filtered data to a CSV file
data.to_csv(filtered_csv_path, index=False)

# If filtered CSV exists, just load it
else:
    data = pd.read_csv(filtered_csv_path)

data = data.sample(n=100000, random_state=1, replace=False)
data.reset_index(drop=True, inplace=True)

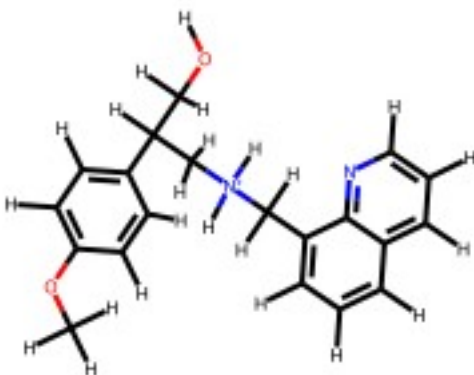
# Save the subsample to a CSV file
data.to_csv("/home/ec2-user/MedGAN/datasets/zinc15_quinolines_data/
non_duplicate_filtered_quinolines_zinc15_50atoms.csv", index=False)

# print a sample from the dataset

smiles_test = data['smiles'][100]
print("SMILES:", smiles_test)
molecule = Chem.MolFromSmiles(smiles_test)
molecule = Chem.AddHs(molecule)
print("Num heavy atoms:", molecule.GetNumHeavyAtoms())
molecule

SMILES: [H]OC([H])([H])C([H])(c1c([H])c([H])c(OC([H])([H])
[H])c([H])c1[H])C([H])([H])[N+]( [H])([H])C([H])
([H])c1c([H])c([H])c([H])c2c([H])c([H])c([H])nc12
Num heavy atoms: 24

```



Convert SMILES to Graph representation

This code provides a suite of tools for representing molecules as graphs and vice versa. The main components are:

- Mappings: Dictionaries (atom_mapping, bond_mapping, charge_mapping) are used to translate atom and bond types between their string names and numerical indices. Conversion Functions:
- smiles_to_graph(): Converts SMILES strings into graph matrices.
- graph_to_molecule(): Reconstructs molecules from their graph matrices.
- graph_to_networkx(): Transforms graph matrices into NetworkX graph objects for visualization or algorithmic analysis.
- Visualization: plot_graph() visually represents a molecule using the NetworkX graph, with atoms and bonds color-coded.
- Data Preprocessing: Molecular data is chunked and processed in parallel to convert batches of SMILES strings into graph representations, which are then saved as compressed files for efficiency.

If molecules are already converted to graphs and saved, will be loaded from compressed files.

Overall, these tools facilitate the transition between chemical molecular structures and their graph representations, offering a foundation for graph-based molecular analyses or neural networks.

```
atom_mapping = {
    "C": 0,
    "N": 1,
    "O": 2,
    "H": 3,
    "F": 4,
    "S": 5,
    "Cl": 6,
}

bond_mapping = {
    "SINGLE": 0,
    0: Chem.BondType.SINGLE,
    "DOUBLE": 1,
    1: Chem.BondType.DOUBLE,
    "TRIPLE": 2,
    2: Chem.BondType.TRIPLE,
    "AROMATIC": 3,
    3: Chem.BondType.AROMATIC,
}

charge_mapping = {
    -1: 7,
    1: 8,
}
```

```

NUM_ATOMS = 50
ATOM_DIM = 11 # 7 atom types + 2 for chirality and 2 for charge
BOND_DIM = 4 + 1
LATENT_DIM = 256

# Convert to bidirectional mapping for simplicity
atom_mapping.update({v: k for k, v in atom_mapping.items()})

def graph_to_molecule(graph):
    adjacency, features = graph
    molecule = Chem.RWMol()

    # Filter out 'padding' atoms and bonds
    keep_idx = np.where(
        (np.sum(features[:, :ATOM_DIM], axis=1) != 0) # not a
        'padding' atom
        & (np.sum(adjacency[:, -1], axis=(0, 1)) != 0) # connected to
        something
    )[0]

    features = features[keep_idx]
    adjacency = adjacency[:, keep_idx, :][:, :, keep_idx]

    num_neighbors = np.sum(adjacency, axis=(0, 2))

    for i in range(len(keep_idx)):
        atom_type_idx = np.argmax(features[i, 0:7])
        atom_type = atom_mapping[atom_type_idx]
        atom = Chem.Atom(atom_type)

        # Handle charge
        charge_features = features[i, 7:9]
        atom_charge = None
        for charge_value, charge_idx in charge_mapping.items():
            if charge_features[charge_idx - 7] == 1.0: # Adjusting
                index for subset
                atom_charge = charge_value
                break

        if atom_charge is not None:
            atom.SetFormalCharge(atom_charge)

        # Handle chirality with neighbor check
        if num_neighbors[i] >= 3: # only consider atoms with at least
            3 neighbors
            if features[i, 9] == 1:
                atom.SetChiralTag(Chem.ChiralType.CHI_TETRAHEDRAL_CCW)
            elif features[i, 10] == 1:
                atom.SetChiralTag(Chem.ChiralType.CHI_TETRAHEDRAL_CW)

```

```

    _ = molecule.AddAtom(atom)

    (bonds_ij, atoms_i, atoms_j) = np.where(np.triu(adjacency) == 1)
    for (bond_ij, atom_i, atom_j) in zip(bonds_ij, atoms_i, atoms_j):
        if atom_i == atom_j or bond_ij == BOND_DIM - 1:
            continue
        bond_type = bond_mapping[bond_ij]
        molecule.AddBond(int(atom_i), int(atom_j), bond_type)

    # Sanitize the molecule
    try:
        # Attempt to sanitize the molecule
        flag = Chem.SanitizeMol(molecule, catchErrors=True)

        # Check the flag returned by the sanitization
        if flag != Chem.SanitizeFlags.SANITIZE_NONE:
            return None

    except AtomValenceException as e:
        print(f"AtomValenceException during molecule sanitization: {e}")
        return None
    except ValueError as e:
        if "Violation occurred" in str(e) and "MolFileStereochem.cpp"
in str(e):
            print(f"Chirality error during molecule sanitization: {e}")
            return None
    except Exception as e:
        print(f"Unexpected error during molecule sanitization: {e}")
        return None

    return molecule

```

This code sample is set for a subsample of 100,000 graphs already converted from molecules.

```

save_folder =
"/home/ec2-user/MedGAN/datasets/zinc15_quinolines_data/data_zinc15_sub
set-iii/archive"
chunk_size = 100000

num_chunks = len(data) // chunk_size
#print(num_chunks)

def process_chunk(i):
    adjacency_tensor_file = os.path.join(save_folder,
f'adjacency_tensor_{i}.npz')
    feature_tensor_file = os.path.join(save_folder,
f'feature_tensor_{i}.npz')

```

```

# If tensors exist, load them (decompressed)
adjacency_tensor = np.load(adjacency_tensor_file)['arr_0']
feature_tensor = np.load(feature_tensor_file)['arr_0']

print(f"Loaded tensors from chunk {i}")

return adjacency_tensor, feature_tensor

adjacency_tensors = []
feature_tensors = []

# Process each chunk individually after loading
for i in tqdm(range(num_chunks), desc='Loading data chunks'):
    adjacency_tensor, feature_tensor = process_chunk(i)
    adjacency_tensors.append(adjacency_tensor)
    feature_tensors.append(feature_tensor)

adjacency_tensor = np.concatenate(adjacency_tensors)
feature_tensor = np.concatenate(feature_tensors)

print("adjacency_tensor.shape =", adjacency_tensor.shape)
print("feature_tensor.shape =", feature_tensor.shape)

Loading data chunks: 100%|██████████| 1/1 [00:10<00:00, 10.31s/it]

Loaded tensors from chunk 0

adjacency_tensor.shape = (100000, 5, 50, 50)
feature_tensor.shape = (100000, 50, 11)

```

WGAN R-GCN implementation

This code provides a detailed walkthrough of the implementation of WGAN R-GCN using the Keras library. We break down the components into two main sections: the Graph Generator and the Graph Discriminator.

The Graph Generator function, `GraphGenerator`, is responsible for creating a graph representation of a molecule given latent space inputs.

The Graph Discriminator function, `GraphDiscriminator`, assesses the "realness" of a molecule graph. It uses graph convolution layers to process the adjacency and feature matrices, and provides a scalar output representing the authenticity of the input graph.

```

def GraphGenerator(
    dense_units, dropout_rate, latent_dim, adjacency_shape,
    feature_shape, l1_value=0.1
):

```

```

# placeholder for the AutoEncoder

z = keras.layers.Input(shape=(LATENT_DIM,))
# Propagate through one or more densely connected layers
x = z
for units in dense_units:
    #x = keras.layers.BatchNormalization()(x) #
    x = keras.layers.Dense(units, activation="tanh")(x) #,
kernel_regularizer=regularizers.l1(l1_value)
    x = keras.layers.Dropout(dropout_rate)(x)

# Map outputs of previous layer (x) to [continuous] adjacency
tensors (x_adjacency)
x_adjacency =
keras.layers.Dense(tf.math.reduce_prod(adjacency_shape))(x)
x_adjacency = keras.layers.Reshape(adjacency_shape)(x_adjacency)
# Symmetrify tensors in the last two dimensions
x_adjacency = (x_adjacency + tf.transpose(x_adjacency, (0, 1, 3,
2))) / 2
x_adjacency = keras.layers.Softmax(axis=1)(x_adjacency)

# Map outputs of previous layer (x) to [continuous] feature
tensors (x_features)
x_features =
keras.layers.Dense(tf.math.reduce_prod(feature_shape))(x)
x_features = keras.layers.Reshape(feature_shape)(x_features)
x_features = keras.layers.Softmax(axis=2)(x_features)

return keras.Model(inputs=z, outputs=[x_adjacency, x_features],
name="Generator")

generator = GraphGenerator(
    dense_units=[128, 256, 512, 1024, 2048, 4096],
    dropout_rate=0.10,
    latent_dim=LATENT_DIM,
    adjacency_shape=(BOND_DIM, NUM_ATOMS, NUM_ATOMS),
    feature_shape=(NUM_ATOMS, ATOM_DIM),
)
generator.summary()

# Save the model summary to a text file
with open('generator_summary_model3_zinc15_iii.txt', 'w') as f:
    generator.summary(print_fn=lambda x: f.write(x + '\n'))

# Save the model structure as an image
plot_model(generator, to_file='generator_model3_zinc15_iii.png',
show_shapes=True, show_layer_names=True)

2025-03-04 16:42:26.594743: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996]

```

successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at

<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>

Model: "Generator"

Layer (type) Connected to	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 256)]	0 []
dense (Dense) ['input_1[0][0]']	(None, 128)	32896
dropout (Dropout) ['dense[0][0]']	(None, 128)	0
dense_1 (Dense) ['dropout[0][0]']	(None, 256)	33024
dropout_1 (Dropout) ['dense_1[0][0]']	(None, 256)	0
dense_2 (Dense) ['dropout_1[0][0]']	(None, 512)	131584
dropout_2 (Dropout) ['dense_2[0][0]']	(None, 512)	0
dense_3 (Dense) ['dropout_2[0][0]']	(None, 1024)	525312
dropout_3 (Dropout) ['dense_3[0][0]']	(None, 1024)	0
dense_4 (Dense)	(None, 2048)	2099200

```
['dropout_3[0][0]']
```

dropout_4 (Dropout)	(None, 2048)	0
---------------------	--------------	---

```
['dense_4[0][0]']
```

dense_5 (Dense)	(None, 4096)	8392704
-----------------	--------------	---------

```
['dropout_4[0][0]']
```

```
2025-03-04 16:42:27.158101: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996]
successful NUMA node read from SysFS had negative value (-1), but
there must be at least one NUMA node, so returning NUMA node zero. See
more at
```

```
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/
sysfs-bus-pci#L344-L355
```

```
2025-03-04 16:42:27.160201: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996]
successful NUMA node read from SysFS had negative value (-1), but
there must be at least one NUMA node, so returning NUMA node zero. See
more at
```

```
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/
sysfs-bus-pci#L344-L355
```

```
2025-03-04 16:42:27.163238: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996]
successful NUMA node read from SysFS had negative value (-1), but
there must be at least one NUMA node, so returning NUMA node zero. See
more at
```

```
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/
sysfs-bus-pci#L344-L355
```

```
2025-03-04 16:42:27.164828: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996]
successful NUMA node read from SysFS had negative value (-1), but
there must be at least one NUMA node, so returning NUMA node zero. See
more at
```

```
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/
sysfs-bus-pci#L344-L355
```

```
2025-03-04 16:42:27.166459: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996]
successful NUMA node read from SysFS had negative value (-1), but
there must be at least one NUMA node, so returning NUMA node zero. See
more at
```

```
https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/
sysfs-bus-pci#L344-L355
```

```
2025-03-04 16:42:28.961589: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996]
successful NUMA node read from SysFS had negative value (-1), but
there must be at least one NUMA node, so returning NUMA node zero. See
more at
```

<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
 2025-03-04 16:42:28.963568: I
 tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at
<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
 2025-03-04 16:42:28.965171: I
 tensorflow/compiler/xla/stream_executor/cuda/cuda_gpu_executor.cc:996] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at
<https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
 2025-03-04 16:42:28.966643: I
 tensorflow/core/common_runtime/gpu/gpu_device.cc:1635] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 20560 MB memory: -> device: 0, name: NVIDIA A10G, pci bus id: 0000:00:1e.0, compute capability: 8.6

dropout_5 (Dropout)	(None, 4096)	0
['dense_5[0][0]']		
dense_6 (Dense)	(None, 12500)	51212500
['dropout_5[0][0]']		
reshape (Reshape)	(None, 5, 50, 50)	0
['dense_6[0][0]']		
tf.compat.v1.transpose (TFOpLa	(None, 5, 50, 50)	0
['reshape[0][0]']		
mbda)		
tf.__operators__.add (TFOpLamb	(None, 5, 50, 50)	0
['reshape[0][0]',		
da)		
'tf.compat.v1.transpose[0][0]']		
dense_7 (Dense)	(None, 550)	2253350
['dropout_5[0][0]']		

```
tf.math.truediv (TFOpLambda)      (None, 5, 50, 50)      0  
['tf.__operators__.add[0][0]']
```

```
reshape_1 (Reshape)               (None, 50, 11)         0  
['dense_7[0][0]']
```

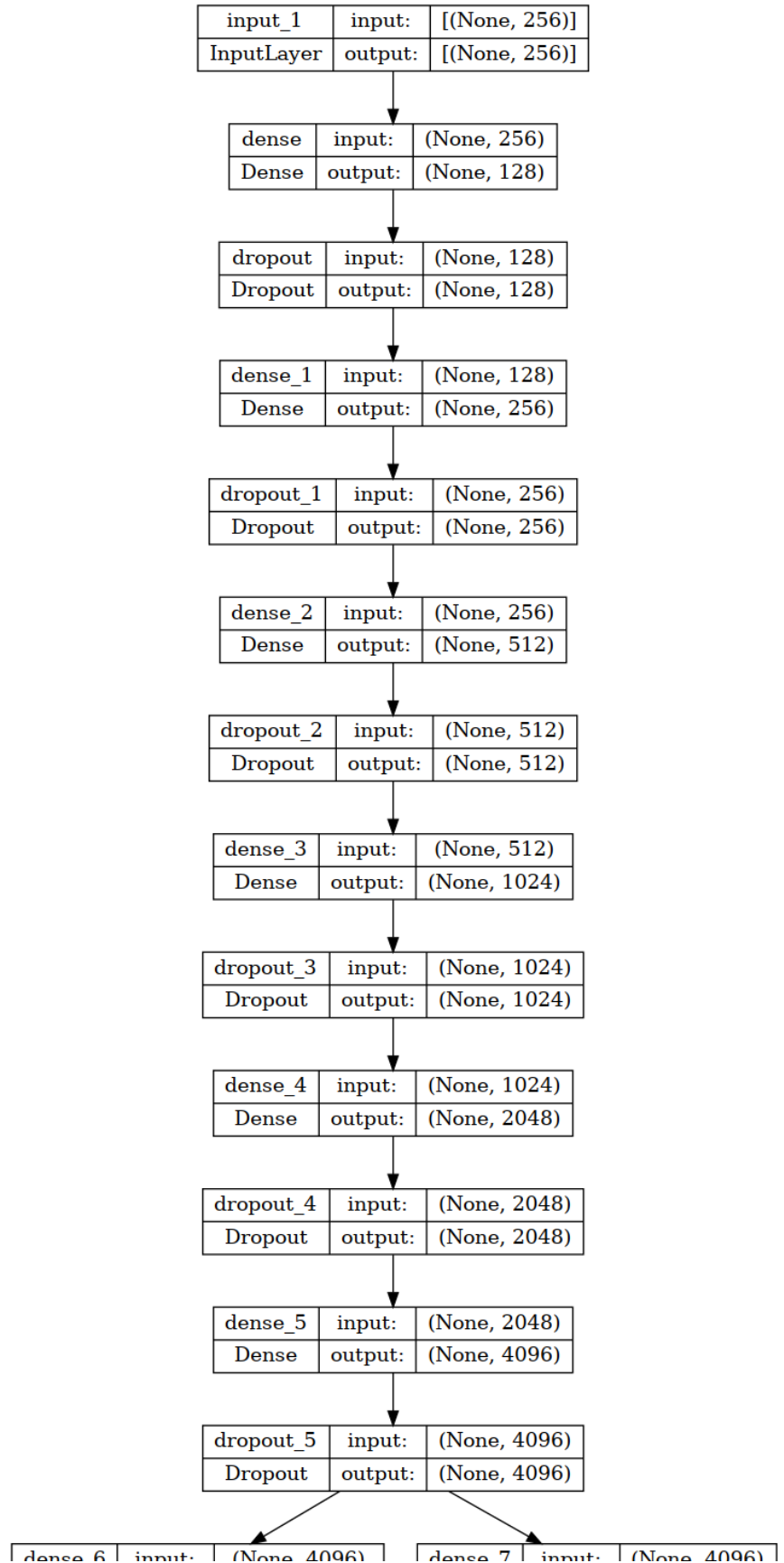
```
softmax (Softmax)                 (None, 5, 50, 50)      0  
['tf.math.truediv[0][0]']
```

```
softmax_1 (Softmax)               (None, 50, 11)         0  
['reshape_1[0][0]']
```

```
=====
```

=====
Total params: 64,680,570
Trainable params: 64,680,570
Non-trainable params: 0

```
=====
```



```

class RelationalGraphConvLayer(keras.layers.Layer):
    def __init__(
        self,
        units=128, # 128
        activation="relu",
        use_bias=False,
        kernel_initializer="glorot_uniform",
        bias_initializer="zeros",
        kernel_regularizer=None,
        #kernel_regularizer=regularizers.l1(0.01), #None,
        bias_regularizer=None,
        **kwargs
    ):
        super().__init__(**kwargs)

        self.units = units
        self.activation = keras.activations.get(activation)
        self.use_bias = use_bias
        self.kernel_initializer =
keras.initializers.get(kernel_initializer)
        self.bias_initializer =
keras.initializers.get(bias_initializer)
        self.kernel_regularizer =
keras.regularizers.get(kernel_regularizer)
        self.bias_regularizer =
keras.regularizers.get(bias_regularizer)

    def build(self, input_shape):
        bond_dim = input_shape[0][1]
        atom_dim = input_shape[1][2]

        self.kernel = self.add_weight(
            shape=(bond_dim, atom_dim, self.units),
            initializer=self.kernel_initializer,
            regularizer=self.kernel_regularizer,
            trainable=True,
            name="W",
            dtype=tf.float32,
        )

        if self.use_bias:
            self.bias = self.add_weight(
                shape=(bond_dim, 1, self.units),
                initializer=self.bias_initializer,
                regularizer=self.bias_regularizer,
                trainable=True,
                name="b",
                dtype=tf.float32,
            )

```

```

        self.built = True

    def call(self, inputs, training=False):
        adjacency, features = inputs
        # Aggregate information from neighbors
        x = tf.matmul(adjacency, features[:, None, :, :])
        # Apply linear transformation
        x = tf.matmul(x, self.kernel)
        if self.use_bias:
            x += self.bias
        # Reduce bond types dim
        x_reduced = tf.reduce_sum(x, axis=1)
        # Apply non-linear transformation
        return self.activation(x_reduced)

def GraphDiscriminator(
    gconv_units, dense_units, dropout_rate, adjacency_shape,
    feature_shape, l1_value=0.01
):
    adjacency = keras.layers.Input(shape=adjacency_shape)
    features = keras.layers.Input(shape=feature_shape)

    # Propagate through one or more graph convolutional layers
    features_transformed = features
    for units in gconv_units:
        features_transformed = RelationalGraphConvLayer(units)(
            [adjacency, features_transformed]
        )

    # Reduce 2-D representation of molecule to 1-D
    x = keras.layers.GlobalAveragePooling1D()(features_transformed)

    # Propagate through one or more densely connected layers
    for units in dense_units:
        # x = keras.layers.Dense(units, activation="relu")(x)
        x = keras.layers.Dense(units, activation="relu")(x) #,
        kernel_regularizer=regularizers.l1(l1_value)
        x = keras.layers.Dropout(dropout_rate)(x)

    # For each molecule, output a single scalar value expressing the
    "realness" of the inputted molecule
    x_out = keras.layers.Dense(1, dtype="float32")(x)

    return keras.Model(inputs=[adjacency, features], outputs=x_out,
        name='Discriminator')

discriminator = GraphDiscriminator(
    gconv_units= [512, 512, 512, 512],

```

```

    dense_units= [4096, 4096],
    dropout_rate=0.10,
    adjacency_shape=(BOND_DIM, NUM_ATOMS, NUM_ATOMS),
    feature_shape=(NUM_ATOMS, ATOM_DIM),
)
discriminator.summary()

# Save the model summary to a text file
with open('discriminator_summary_model3_zinc15_iii.txt', 'w') as f:
    discriminator.summary(print_fn=lambda x: f.write(x + '\n'))

# Save the model structure as an image
plot_model(discriminator,
to_file='discriminator_model3_zinc15_iii.png', show_shapes=True,
show_layer_names=True)

```

Model: "Discriminator"

Layer (type) Connected to	Output Shape	Param #
=====		
input_2 (InputLayer)	[(None, 5, 50, 50)]	0
input_3 (InputLayer)	[(None, 50, 11)]	0
relational_graph_conv_layer (RelationalGraphConvLayer) ['input_2[0][0]', 'input_3[0][0]']	(None, 50, 512)	28160
relational_graph_conv_layer_1 (RelationalGraphConvLayer) ['input_2[0][0]', 'relational_graph_conv_layer[0][0]']	(None, 50, 512)	1310720
relational_graph_conv_layer_2 (RelationalGraphConvLayer) ['input_2[0][0]', 'relational_graph_conv_layer_1[0][0]']	(None, 50, 512)	1310720

```
[0]']
```

```
relational_graph_conv_layer_3 (None, 50, 512) 1310720  
['input_2[0][0]',  
 (RelationalGraphConvLayer)  
'relational_graph_conv_layer_2[0  
[0]']
```

```
global_average_pooling1d (Glob (None, 512) 0  
['relational_graph_conv_layer_3[0  
alAveragePooling1D)  
[0]']
```

```
dense_8 (Dense) (None, 4096) 2101248  
['global_average_pooling1d[0][0]'
```

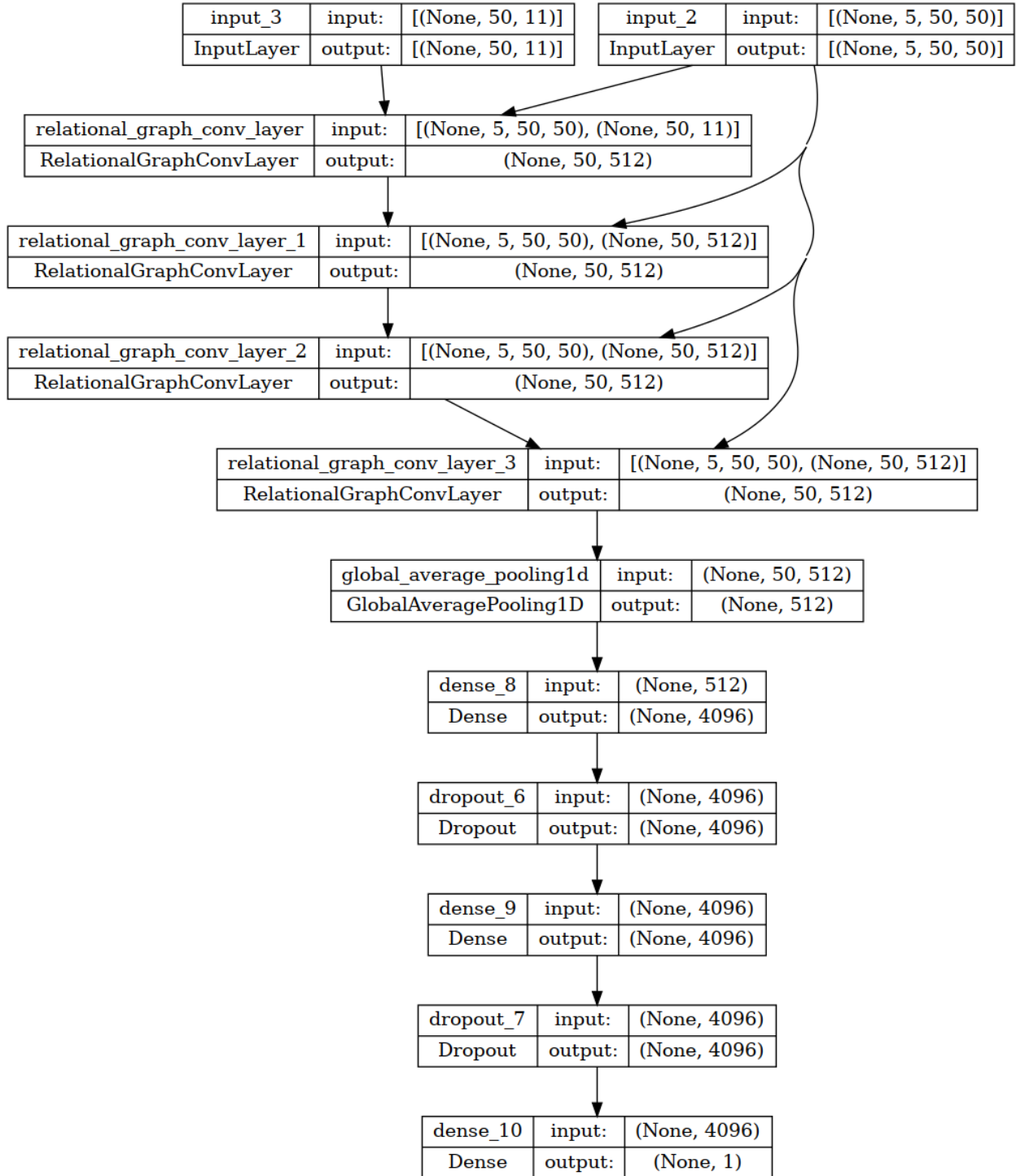
```
dropout_6 (Dropout) (None, 4096) 0  
['dense_8[0][0]']
```

```
dense_9 (Dense) (None, 4096) 16781312  
['dropout_6[0][0]']
```

```
dropout_7 (Dropout) (None, 4096) 0  
['dense_9[0][0]']
```

```
dense_10 (Dense) (None, 1) 4097  
['dropout_7[0][0]']
```

```
=====  
=====  
Total params: 22,846,977  
Trainable params: 22,846,977  
Non-trainable params: 0
```



Generative Adversarial Network for Graphs

The GraphWGAN class implements a Generative Adversarial Network (GAN) to generate graphs. This GAN leverages Wasserstein distance with gradient penalty for training stability.

The `train_step` method trains the GAN for one step. This involves training the discriminator to differentiate between real and generated graphs, and training the generator to produce graphs that the discriminator cannot differentiate from real graphs.

```
class GraphWGAN(keras.Model):
    def __init__(
        self,
        generator,
        discriminator,
        discriminator_steps=1,
        generator_steps=1,
        gp_weight=10,
        **kwargs
    ):
        super().__init__(**kwargs)
        self.generator = generator
        self.discriminator = discriminator
        self.discriminator_steps = discriminator_steps
        self.generator_steps = generator_steps
        self.gp_weight = gp_weight
        self.latent_dim = self.generator.input_shape[-1]
        self.epoch = 0
        self.batch = 0
        self.quinoline_scaffold = Chem.MolFromSmiles("n1cccc2ccccc12")

        # We will store our dynamic "reward" scalars here
        self.value_for_zeros = 1.0
        self.value_for_ones = 10.0 # Start at 10 and we will
        adaptively increase it.

        # Initialize metrics for the logs
        self.metric_loss_gp = keras.metrics.Mean(name='loss_gp')
        self.metric_discriminator_loss =
keras.metrics.Mean(name='discriminator_loss')
        self.metric_disc_loss = keras.metrics.Mean(name='disc_loss')
        self.metric_rewards = keras.metrics.Mean(name='rewards')
        self.metric_connectivity_g =
keras.metrics.Mean(name='connectivity_g')
        self.metric_gen_loss = keras.metrics.Mean(name='gen_loss')
        self.metric_critic_g_fake =
keras.metrics.Mean(name='critic_g_fake')
        self.metric_critic_g_fake_loss =
keras.metrics.Mean(name='critic_g_fake_loss')

        def compile(self, optimizer_generator, optimizer_discriminator,
**kwargs):
            super().compile(**kwargs)
            self.optimizer_generator = optimizer_generator
            self.optimizer_discriminator = optimizer_discriminator
```

```

# code to generate samples and rewards
def z_to_mol(self, graph, sample_size):
    adjacency = tf.argmax(graph[0], axis=1)
    adjacency = tf.one_hot(adjacency, depth=BOND_DIM, axis=1)
    adjacency = tf.linalg.set_diag(adjacency,
tf.zeros(tf.shape(adjacency)[:,-1]))
    features = tf.argmax(graph[1], axis=2)
    features = tf.one_hot(features, depth=ATOM_DIM, axis=2)
    molecules = []
    for i in range(sample_size):    # for i in range(sample_size):
        try:
            mol = graph_to_molecule([adjacency[i].numpy(),
features[i].numpy()])
            if mol:
                molecules.append(mol)
            else:
                molecules.append(None)
        except AtomValenceException as e:
            logging.error(f"AtomValenceException during molecule
generation: {e}")
            molecules.append(None)
        except ValueError as e:
            if "Violation occurred" in str(e) and
"MolFileStereochem.cpp" in str(e):
                logging.error(f"Chirality error during molecule
generation: {e}")
                molecules.append(None)
            else:
                logging.error(f"Unexpected ValueError during
molecule generation: {e}")
                molecules.append(None)
        except RuntimeError as e:
            logging.error(f"RuntimeError during molecule
generation: {e}")
            molecules.append(None)
        except Exception as e:
            logging.error(f"Unexpected error during molecule
generation: {e}")
            molecules.append(None)
    return molecules

# code to train
def train_step(self, inputs):
    start_time = time.time()
    noise_dim = self.generator.input_shape[1]

    if isinstance(inputs[0], tuple):
        inputs = inputs[0]

    real_graphs = inputs

```

```

self.batch_size = tf.shape(inputs[0])[0]

def individual_metric_validity(molecule):
    if molecule is None or molecule.GetNumAtoms() == 0:
        return 0
    try:
        mol_block = Chem.MolToMolBlock(molecule)
        if not all(char == '0' for char in mol_block):
            num_frags = len(rdmolops.GetMolFrags(molecule,
asMols=False, sanitizeFrags=False))
            if num_frags == 1:
                return 1
            else:
                return 0
    except Exception:
        return 0

# Training the discriminator (critic)
for _ in range(self.discriminator_steps):

    with tf.GradientTape() as disc_tape:

        z = tf.random.normal([self.batch_size,
self.latent_dim])
        generated_d_graphs = self.generator(z, training=True)

        critic_real = self.discriminator(real_graphs,
training=True)
        critic_fake = self.discriminator(generated_d_graphs,
training=True)

        # Calculate gradient penalty
        loss_gp = self._gradient_penalty(real_graphs,
generated_d_graphs)
        discriminator_loss = tf.reduce_mean(critic_fake) -
tf.reduce_mean(critic_real)
        disc_loss = discriminator_loss + loss_gp *
self.gp_weight

        gradients_of_discriminator = disc_tape.gradient(disc_loss,
self.discriminator.trainable_variables)

self.optimizer_discriminator.apply_gradients(zip(gradients_of_discrimi
nator, self.discriminator.trainable_variables))

    # Update the metric values
    self.metric_loss_gp.update_state(loss_gp)

self.metric_discriminator_loss.update_state(discriminator_loss)
self.metric_disc_loss.update_state(disc_loss)

```

```

    # Training the generator
    for _ in range(self.generator_steps):
        with tf.GradientTape() as gen_tape:
            # Generate fake data here
            z = tf.random.normal([self.batch_size,
self.latent_dim])
            generated_g_graphs = self.generator(z, training=True)

            # Convert generated graphs to molecules for reward
            calculation
            generated_g_molecules =
self.z_to_mol(graph=generated_g_graphs, sample_size=self.batch_size)

            critic_g_fake = self.discriminator(generated_g_graphs,
training=True)
            critic_g_fake_tensor =
tf.convert_to_tensor(critic_g_fake, dtype=tf.float32)
            critic_g_fake_loss = -tf.reduce_mean(critic_g_fake)

            # Calculate rewards based on molecule connectivity
            rewards = [individual_metric_validity(mol) for mol in
generated_g_molecules]
            rewards_tensor = tf.convert_to_tensor(rewards,
dtype=tf.float32)

            connectivity_g_numpy = np.array(rewards)
            connectivity_g = np.mean(connectivity_g_numpy)

            # We use our dynamic "value_for_zeros" and
            "value_for_ones"
            scaled_rewards_tensor = tf.where(
                rewards_tensor == 0,
                self.value_for_zeros, # e.g. 1.0
                self.value_for_ones # e.g. 10.0, adaptively
updated
            )

            # Reshape scaled rewards for element-wise
            multiplication with critic scores
            adjusted_critic_scores = critic_g_fake_tensor *
tf.reshape(scaled_rewards_tensor, (-1, 1))
            adjusted_critic_scores_loss = -
tf.reduce_mean(adjusted_critic_scores)

            lambda_reward = 1.0

            # Calculate the generator loss
            gen_loss = lambda_reward * adjusted_critic_scores_loss
            # -tf.reduce_mean(adjusted_critic_scores)

```

```

        # Apply generator gradients
        gradients_of_generator = gen_tape.gradient(gen_loss,
self.generator.trainable_variables)

        # Apply the clipped gradients

self.optimizer_generator.apply_gradients(zip(gradients_of_generator,
self.generator.trainable_variables))

        # Update the metric values
self.metric_rewards.update_state(rewards)
self.metric_connectivity_g.update_state(connectivity_g)
self.metric_gen_loss.update_state(gen_loss)
self.metric_critic_g_fake.update_state(critic_g_fake)

self.metric_critic_g_fake_loss.update_state(critic_g_fake_loss)

        # Increment the epoch counter
self.epoch += 1

        end_time = time.time()
        time_per_epoch = end_time - start_time

        logs = {m.name: m.result() for m in self.metrics}
        logs['time'] = time_per_epoch

        return logs

def _gradient_penalty(self, graph_real, graph_generated):
    # Unpack graphs
    adjacency_real, features_real = graph_real
    adjacency_generated, features_generated = graph_generated

    # Generate interpolated graphs (adjacency_interp and
features_interp)
    alpha = tf.random.uniform([self.batch_size])
    alpha = tf.reshape(alpha, (self.batch_size, 1, 1, 1))
    adjacency_interp = (adjacency_real * alpha) + (1 - alpha) *
adjacency_generated
    alpha = tf.reshape(alpha, (self.batch_size, 1, 1))
    features_interp = (features_real * alpha) + (1 - alpha) *
features_generated

    # Compute the logits of interpolated graphs
    with tf.GradientTape() as tape:
        tape.watch(adjacency_interp)
        tape.watch(features_interp)
        logits = self.discriminator(
            [adjacency_interp, features_interp], training=True

```

```

    )

    # Compute the gradients with respect to the interpolated graphs
    grads = tape.gradient(logits, [adjacency_interp,
    features_interp])
    # Compute the gradient penalty
    grads_adjacency_penalty = (1 - tf.norm(grads[0], axis=1)) ** 2
    grads_features_penalty = (1 - tf.norm(grads[1], axis=2)) ** 2
    return tf.reduce_mean(
        tf.reduce_mean(grads_adjacency_penalty, axis=(-2, -1))
        + tf.reduce_mean(grads_features_penalty, axis=(-1))
    )

    def save_model(self,
    folder_path="training_models_model3_zinc15_iii/WGAN"):
        if not os.path.exists(folder_path):
            os.makedirs(folder_path)
        self.generator.save(os.path.join(folder_path, "generator"))
        self.discriminator.save(os.path.join(folder_path,
        "discriminator"))

    def load_model(self,
    folder_path="training_models_model3_zinc15_iii/WGAN"):
        self.generator =
        keras.models.load_model(os.path.join(folder_path, "generator"))
        self.discriminator =
        keras.models.load_model(os.path.join(folder_path, "discriminator"))

```

Training metrics

This code is a detailed implementation of logging various metrics on Tensorboard related to the progress and performance of a Generative Adversarial Network (GAN) designed to generate molecular structures. It computes metrics such as validity, uniqueness, novelty, quinoline, and scaffold similarity. But also training performance on weights and layers.

```

# Load the full quinoline dataset
quinolines_df =
pd.read_csv("/home/ec2-user/MedGAN/datasets/zinc15_quinolines_data/
non_duplicate_filtered_quinolines_zinc15_50atoms.csv")
full_quinolines_smiles = set(quinolines_df['smiles'].tolist())

class GANLogger(tf.keras.callbacks.Callback):
    def __init__(self, tensorboard_logdir, original_dataset,
    full_dataset, num_samples):
        super().__init__()
        self.tensorboard_logdir = tensorboard_logdir
        self.writer =
        tf.summary.create_file_writer(tensorboard_logdir)

```

```

self.original_smiles = set(original_dataset)
self.full_quinolines_smiles = set(full_dataset)
self.num_samples = num_samples
self.quinoline_scaffold = Chem.MolFromSmiles("n1cccc2ccccc12")
self.best_connected_validity = 0.0
self.epochs_without_improvement = 0

def mol_sample(self, generator, batch_size):
    z = tf.random.normal((batch_size, LATENT_DIM))
    graph = generator.predict(z)
    adjacency = tf.argmax(graph[0], axis=1)
    adjacency = tf.one_hot(adjacency, depth=BOND_DIM, axis=1)
    adjacency = tf.linalg.set_diag(adjacency,
tf.zeros(tf.shape(adjacency)[: -1]))
    features = tf.argmax(graph[1], axis=2)
    features = tf.one_hot(features, depth=ATOM_DIM, axis=2)
    molecules = []
    none_counter = 0
    for i in tqdm(range(batch_size), desc="Generating molecules"):
        try:
            mol = graph_to_molecule([adjacency[i].numpy(),
features[i].numpy()])
            if mol:
                molecules.append(mol)
            else:
                molecules.append(None)
                none_counter += 1
        except AtomValenceException as e:
            logging.error(f"AtomValenceException during molecule
generation: {e}")
            molecules.append(None)
            none_counter += 1
        except ValueError as e:
            if "Violation occurred" in str(e) and
"MolFileStereochem.cpp" in str(e):
                logging.error(f"Chirality error during molecule
generation: {e}")
                molecules.append(None)
                none_counter += 1
            else:
                logging.error(f"Unexpected ValueError during
molecule generation: {e}")
                molecules.append(None)
                none_counter += 1
        except RuntimeError as e:
            logging.error(f"RuntimeError during molecule
generation: {e}")
            molecules.append(None)
            none_counter += 1

```

```

        except Exception as e:
            logging.error(f"Unexpected error during molecule
generation: {e}")
            molecules.append(None)
            none_counter += 1 # Increment the counter
        return molecules, none_counter # Return the counter

    def on_epoch_begin(self, epoch, logs=None):
        if epoch == 0: # Just trace the graph once, at the beginning
            tf.summary.trace_on(graph=True, profiler=True)

    def on_epoch_end(self, epoch, logs=None):
        logs = logs or {}
        with self.writer.as_default():
            # Scalars
            tf.summary.scalar('loss_gp', logs.get("loss_gp"),
step=epoch)
            tf.summary.scalar('discriminator_loss',
logs.get("discriminator_loss"), step=epoch)
            tf.summary.scalar('disc_loss', logs.get("disc_loss"),
step=epoch)
            tf.summary.scalar('rewards', logs.get("rewards"),
step=epoch)
            tf.summary.scalar('connectivity_g',
logs.get("connectivity_g"), step=epoch)
            tf.summary.scalar('gen_loss', logs.get("gen_loss"),
step=epoch)
            tf.summary.scalar('critic_g_fake_loss',
logs.get("critic_g_fake_loss"), step=epoch)
            tf.summary.scalar('critic_g_fake',
logs.get("critic_g_fake"), step=epoch)

            # Time per epoch
            if 'time' in logs:
                tf.summary.scalar("time_per_epoch", logs.get("time"),
step=epoch)

            # Histograms of trainable variables
            for var in self.model.trainable_variables:
                tf.summary.histogram(var.name, var, step=epoch)

            self.writer.flush()

        if epoch == 0: # Write the graph at the end of the first
epoch
            with self.writer.as_default():
                tf.summary.trace_export(name="model_trace",
step=epoch, profiler_outdir=self.tensorboard_logdir)

```

```

        # Evaluate model's generation metrics every epoch or every N
epochs
        molecules, none_counter =
self.mol_sample(self.model.generator, batch_size=100)
        none_percentage = none_counter / 100

        valid_molecules = []
        for m in molecules:
            if m is not None and m.GetNumAtoms() > 0:
                try:
                    mol_block = Chem.MolToMolBlock(m)
                    if not all(char == '0' for char in mol_block):
                        valid_molecules.append(m)
                except Exception:
                    # Silently ignore any errors in block conversion
                    pass

        connected_valid_molecules = [m for m in valid_molecules if
len(GetMolFragments(m)) == 1]
        connected_valid_molecules_with_Hs = [AddHs(mol) for mol in
connected_valid_molecules]

        # Compute Tanimoto similarity to a known scaffold
        similarities = []
        for m in valid_molecules:
            if m.GetNumAtoms() > 0:
                try:
                    scaffold = MurckoScaffold.GetScaffoldForMol(m)
                    similarity = DataStructs.TanimotoSimilarity(
AllChem.GetMorganFingerprint(self.quinoline_scaffold, 2),
AllChem.GetMorganFingerprint(scaffold, 2)
                    )
                    similarities.append(similarity)
                except Chem.rdchem.AtomValenceException:
                    print("Invalid molecule skipped due to
AtomValenceException")

        average_similarity = 0
        if len(valid_molecules) > 0:
            validity = len(valid_molecules) / len(molecules)
            if len(connected_valid_molecules) > 0:
                connected_validity =
len(connected_valid_molecules_with_Hs) / len(molecules)
            else:
                connected_validity = 0

        unique_molecules = list(set(valid_molecules))
        if len(unique_molecules) > 0:
            uniqueness = len(unique_molecules) /

```

```

len(valid_molecules)

        novel_molecules = [
            Chem.MolToSmiles(mol, isomericSmiles=False,
allBondsExplicit=False)
            for mol in unique_molecules
            if Chem.MolToSmiles(mol, isomericSmiles=False,
allBondsExplicit=False)
            not in self.original_smiles
        ]
        novelty = len(novel_molecules) / len(unique_molecules)
if len(unique_molecules) > 0 else 0

        absolute_novel_molecules = [
            Chem.MolToSmiles(mol, isomericSmiles=False,
allBondsExplicit=False)
            for mol in unique_molecules
            if Chem.MolToSmiles(mol, isomericSmiles=False,
allBondsExplicit=False)
            not in self.full_quinolines_smiles
        ]
        absolute_novelty = (
            len(absolute_novel_molecules) /
len(unique_molecules)
            if len(unique_molecules) > 0
            else 0
        )
    else:
        uniqueness = 0
        novelty = 0
        absolute_novelty = 0
    else:
        validity = 0
        novelty = 0
        uniqueness = 0
        connected_validity = 0
        absolute_novelty = 0

    if len(valid_molecules) > 0:
        quinoline_molecules = [mol for mol in valid_molecules if
mol.HasSubstructMatch(self.quinoline_scaffold)]
        quinoline_percentage = len(quinoline_molecules) /
len(valid_molecules)
    else:
        quinoline_percentage = 0

    if len(similarities) > 0:
        average_similarity = sum(similarities) / len(similarities)

    if connected_validity > self.best_connected_validity:

```

```

        self.best_connected_validity = connected_validity
        self.epochs_without_improvement = 0
    else:
        self.epochs_without_improvement += 1
        if self.epochs_without_improvement >= 10:
            self.model.value_for_ones += 10.0
            print(
                f"=== connected_validity plateaued. "
                f"Increasing value_for_ones to
{self.model.value_for_ones} ==="
            )
            self.epochs_without_improvement = 0

    with self.writer.as_default():
        tf.summary.scalar("validity", validity, step=epoch)
        tf.summary.scalar("connected_validity",
connected_validity, step=epoch)
        tf.summary.scalar("uniqueness", uniqueness, step=epoch)
        tf.summary.scalar("novelty", novelty, step=epoch)
        tf.summary.scalar("absolute_novelty", absolute_novelty,
step=epoch)
        tf.summary.scalar("quinoline_percentage",
quinoline_percentage, step=epoch)
        tf.summary.scalar("average_tanimoto_similarity",
average_similarity, step=epoch)
        tf.summary.scalar("none_molecule_percentage",
none_percentage, step=epoch)

        # Optional: Log the current scaling factor as well
        tf.summary.scalar("current_value_for_ones",
self.model.value_for_ones, step=epoch)

    self.writer.flush()

```

Train the model

In the following code, the WGAN is constructed and trained. The PlotSamplesCallback class is a custom Keras callback to visualize and plot samples at the end of each training epoch. This helps in monitoring the evolution of generated structures over the training process. To ensure continuity and robustness in training, checkpoints are used. A data_generator function is defined to produce training batches from provided tensors, ensuring that the data is fed correctly into the GAN during training. The GAN is then trained using the fit method, with checkpoints, sample plotting, and logging utilities as its callbacks. After training is complete, the model is saved, providing a reusable pre-trained model for future tasks.

```

class PlotSamplesCallback(keras.callbacks.Callback):
    def __init__(self, wgan, num_samples=1):
        super().__init__()
        self.wgan = wgan

```

```

        self.num_samples = num_samples

    def on_epoch_end(self, epoch, logs=None):
        if epoch % 1 == 0:
            print("\nGenerating and plotting samples at epoch", epoch)

invalid_mol = MolFromSmiles("")

tensorboard_logdir = "logs_model3_zinc15_iii/WGAN"
log_counter_G = 0
log_counter_D = 0

# Create the GraphWGAN instance and configure it
wgan = GraphWGAN(generator,
                  discriminator,
                  discriminator_steps=1,
                  generator_steps=1,
                  gp_weight=10,
                  )

wgan.compile(
    optimizer_generator=keras.optimizers.RMSprop(1e-4),
    optimizer_discriminator=keras.optimizers.RMSprop(1e-4)
)

# Checkpoint configurations
checkpoint_dir = 'training_checkpoints_model3_zinc15_iii/WGAN'
checkpoint_prefix = os.path.join(checkpoint_dir, 'ckpt')

if not os.path.exists(checkpoint_dir):
    os.makedirs(checkpoint_dir)

checkpoint = tf.train.Checkpoint(
    epoch=tf.Variable(0),
    generator=wgan.generator,
    discriminator=wgan.discriminator,
    optimizer_generator=wgan.optimizer_generator,
    optimizer_discriminator=wgan.optimizer_discriminator,
)

def load_latest_checkpoint(checkpoint):
    latest_checkpoint = tf.train.latest_checkpoint(checkpoint_dir)
    if latest_checkpoint:
        print(f"Loading checkpoint from {latest_checkpoint}")
        checkpoint.restore(latest_checkpoint)
        last_epoch = int(checkpoint.epoch.numpy())
    else:
        print("No checkpoint found. Training from scratch.")
        last_epoch = 0

```

```

    return last_epoch

# Load the latest checkpoint and get the starting epoch
starting_epoch = load_latest_checkpoint(checkpoint)

# Create a PlotSamplesCallback instance
plot_samples_callback = PlotSamplesCallback(wgan, num_samples=1)

class CustomModelCheckpoint(keras.callbacks.Callback):

    def clear_training_molecules_folder(self):
        folder_path = "training_molecules"
        if os.path.exists(folder_path):
            shutil.rmtree(folder_path)
        os.makedirs(folder_path, exist_ok=True)

    def on_epoch_end(self, epoch, logs=None):
        # Update the checkpoint's epoch
        checkpoint.epoch.assign(epoch + 1)
        # Save the checkpoint at the end of each epoch
        checkpoint.save(file_prefix=checkpoint_prefix)

        # Clear the contents of the training_molecules folder
        #self.clear_training_molecules_folder()

        tf.keras.backend.clear_session()
        gc.collect()

checkpoint_callback = CustomModelCheckpoint()

# Instantiate the GANLogger
gan_logger = GANLogger(
    tensorboard_logdir,
    data['smiles'].tolist(),
    full_quinolines_smiles,
    num_samples=1)

Loading checkpoint from
training_checkpoints_model3_zinc15_iii/WGAN/ckpt-400

print(tf.config.list_physical_devices('GPU'))

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

tf.config.run_functions_eagerly(True)
#tf.config.run_functions_eagerly(False)
print("Is TensorFlow running in eager mode:", tf.executing_eagerly())

Is TensorFlow running in eager mode: True

```

```

class ResetMetricsCallback(keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        # Resetting metric states at the end of each epoch
        self.model.metric_loss_gp.reset_states()
        self.model.metric_discriminator_loss.reset_states()
        self.model.metric_disc_loss.reset_states()
        self.model.metric_rewards.reset_states()
        self.model.metric_connectivity_g.reset_states()
        self.model.metric_gen_loss.reset_states()
        self.model.metric_critic_g_fake.reset_states()
        self.model.metric_critic_g_fake_loss.reset_states()

    def data_generator(adjacency_tensor, feature_tensor, batch_size):
        dataset_size = len(adjacency_tensor)
        indices = np.arange(dataset_size)
        while True:
            # Shuffle indices at the start of each epoch
            np.random.shuffle(indices)
            for i in range(0, dataset_size, batch_size):
                batch_indices = indices[i: min(i + batch_size,
dataset_size)]
                batch_adjacency_tensor = adjacency_tensor[batch_indices]
                batch_feature_tensor = feature_tensor[batch_indices]
                yield [batch_adjacency_tensor, batch_feature_tensor]

batch_size = 512 # Set your batch size
data_gen = data_generator(adjacency_tensor, feature_tensor,
batch_size)

steps_per_epoch = len(adjacency_tensor) // batch_size
if len(adjacency_tensor) % batch_size != 0:
    steps_per_epoch += 1

# Train the model
wgan.fit(
    data_gen,
    initial_epoch=starting_epoch,
    epochs= 500,
    steps_per_epoch=steps_per_epoch,
    callbacks=[
        checkpoint_callback,
        plot_samples_callback,
        gan_logger,
        ResetMetricsCallback()
    ],
)
# Save the trained model
wgan.save_model()

```

Epoch 401/500

```

/home/ec2-user/miniconda3/envs/my-rdkit-env/lib/python3.10/site-
packages/tensorflow/python/data/ops/structured_function.py:254:
UserWarning: Even though the
`tf.config.experimental_run_functions_eagerly` option is set, this
option does not apply to tf.data functions. To force eager execution
of tf.data functions, please use
`tf.data.experimental.enable_debug_mode()`.
  warnings.warn(
2025-03-04 16:42:36.961482: I
tensorflow/core/common_runtime/executor.cc:1197] [/device:CPU:0]
(DEBUG INFO) Executor start aborting (this does not indicate an error
and you can ignore this message): INVALID_ARGUMENT: You must feed a
value for placeholder tensor 'Placeholder/_0' with dtype int32
  [[{{node Placeholder/_0}}]]
2025-03-04 16:42:39.075643: I
tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:637]
TensorFloat-32 will be used for the matrix multiplication. This will
only be logged once.

196/196 [=====] - ETA: 0s - loss_gp: 2.0498 -
discriminator_loss: -56.3057 - disc_loss: -35.8074 - rewards: 0.0491 -
connectivity_g: 0.0492 - gen_loss: 4.7913 - critic_g_fake: -2.7839 -
critic_g_fake_loss: 2.7844 - time: 1.5377
Generating and plotting samples at epoch 400

2025-03-04 16:47:40.774628: W
tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
2013265920 exceeds 10% of free system memory.
2025-03-04 16:47:41.540859: W
tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
12288000000 exceeds 10% of free system memory.
2025-03-04 16:47:45.350993: W
tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
4026531840 exceeds 10% of free system memory.

4/4 [=====] - 0s 8ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
525.85it/s]

196/196 [=====] - 310s 2s/step - loss_gp:
2.0498 - discriminator_loss: -56.3057 - disc_loss: -35.8074 - rewards:
0.0491 - connectivity_g: 0.0492 - gen_loss: 4.7913 - critic_g_fake: -
2.7839 - critic_g_fake_loss: 2.7844 - time: 1.5326
Epoch 402/500
196/196 [=====] - ETA: 0s - loss_gp: 2.0347 -
discriminator_loss: -56.1100 - disc_loss: -35.7629 - rewards: 0.0862 -
connectivity_g: 0.0863 - gen_loss: 4.4736 - critic_g_fake: -2.2155 -
critic_g_fake_loss: 2.2757 - time: 1.5568
Generating and plotting samples at epoch 401

```

```
2025-03-04 16:52:54.155665: W
tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
2013265920 exceeds 10% of free system memory.
2025-03-04 16:52:54.881935: W
tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of
12288000000 exceeds 10% of free system memory.

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
542.07it/s]

196/196 [=====] - 313s 2s/step - loss_gp:
2.0347 - discriminator_loss: -56.1100 - disc_loss: -35.7629 - rewards:
0.0862 - connectivity_g: 0.0863 - gen_loss: 4.4736 - critic_g_fake: -
2.2155 - critic_g_fake_loss: 2.2757 - time: 1.5517
Epoch 403/500
196/196 [=====] - ETA: 0s - loss_gp: 2.0235 -
discriminator_loss: -56.1343 - disc_loss: -35.8996 - rewards: 0.0974 -
connectivity_g: 0.0974 - gen_loss: 12.5766 - critic_g_fake: -3.6874 -
critic_g_fake_loss: 3.6897 - time: 1.5717
Generating and plotting samples at epoch 402
4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
509.96it/s]

196/196 [=====] - 316s 2s/step - loss_gp:
2.0235 - discriminator_loss: -56.1343 - disc_loss: -35.8996 - rewards:
0.0974 - connectivity_g: 0.0974 - gen_loss: 12.5766 - critic_g_fake: -
3.6874 - critic_g_fake_loss: 3.6897 - time: 1.5665
Epoch 404/500
196/196 [=====] - ETA: 0s - loss_gp: 2.0408 -
discriminator_loss: -56.3180 - disc_loss: -35.9104 - rewards: 0.0794 -
connectivity_g: 0.0793 - gen_loss: 10.2868 - critic_g_fake: -3.3224 -
critic_g_fake_loss: 3.2875 - time: 1.5695
Generating and plotting samples at epoch 403
4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
552.31it/s]

196/196 [=====] - 315s 2s/step - loss_gp:
2.0408 - discriminator_loss: -56.3180 - disc_loss: -35.9104 - rewards:
0.0794 - connectivity_g: 0.0793 - gen_loss: 10.2868 - critic_g_fake: -
3.3224 - critic_g_fake_loss: 3.2875 - time: 1.5643
Epoch 405/500
196/196 [=====] - ETA: 0s - loss_gp: 2.0460 -
discriminator_loss: -56.3401 - disc_loss: -35.8803 - rewards: 0.0804 -
connectivity_g: 0.0805 - gen_loss: 9.4845 - critic_g_fake: -2.8416 -
critic_g_fake_loss: 3.0061 - time: 1.5564
```

Generating and plotting samples at epoch 404

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
532.11it/s]

Invalid molecule skipped due to AtomValenceException

196/196 [=====] - 313s 2s/step - loss_gp:
2.0460 - discriminator_loss: -56.3401 - disc_loss: -35.8803 - rewards:
0.0804 - connectivity_g: 0.0805 - gen_loss: 9.4845 - critic_g_fake: -
2.8416 - critic_g_fake_loss: 3.0061 - time: 1.5513

Epoch 406/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0319 -
discriminator_loss: -56.1838 - disc_loss: -35.8649 - rewards: 0.0883 -
connectivity_g: 0.0883 - gen_loss: 11.4036 - critic_g_fake: -3.0205 -
critic_g_fake_loss: 3.0060 - time: 1.5622

Generating and plotting samples at epoch 405

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
543.57it/s]

196/196 [=====] - 314s 2s/step - loss_gp:
2.0319 - discriminator_loss: -56.1838 - disc_loss: -35.8649 - rewards:
0.0883 - connectivity_g: 0.0883 - gen_loss: 11.4036 - critic_g_fake: -
3.0205 - critic_g_fake_loss: 3.0060 - time: 1.5571

Epoch 407/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0176 -
discriminator_loss: -55.8207 - disc_loss: -35.6443 - rewards: 0.0852 -
connectivity_g: 0.0852 - gen_loss: 11.7968 - critic_g_fake: -2.9981 -
critic_g_fake_loss: 3.0056 - time: 1.5561

Generating and plotting samples at epoch 406

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
533.69it/s]

196/196 [=====] - 312s 2s/step - loss_gp:
2.0176 - discriminator_loss: -55.8207 - disc_loss: -35.6443 - rewards:
0.0852 - connectivity_g: 0.0852 - gen_loss: 11.7968 - critic_g_fake: -
2.9981 - critic_g_fake_loss: 3.0056 - time: 1.5511

Epoch 408/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0237 -
discriminator_loss: -55.8508 - disc_loss: -35.6142 - rewards: 0.0837 -
connectivity_g: 0.0838 - gen_loss: 8.2293 - critic_g_fake: -2.7278 -
critic_g_fake_loss: 2.7069 - time: 1.5632

Generating and plotting samples at epoch 407

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
520.37it/s]

196/196 [=====] - 314s 2s/step - loss_gp:
2.0237 - discriminator_loss: -55.8508 - disc_loss: -35.6142 - rewards:
0.0837 - connectivity_g: 0.0838 - gen_loss: 8.2293 - critic_g_fake: -
2.7278 - critic_g_fake_loss: 2.7069 - time: 1.5581

Epoch 409/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0392 -
discriminator_loss: -56.1984 - disc_loss: -35.8060 - rewards: 0.0952 -
connectivity_g: 0.0951 - gen_loss: 10.7136 - critic_g_fake: -2.6820 -
critic_g_fake_loss: 2.7202 - time: 1.5628

Generating and plotting samples at epoch 408

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
553.96it/s]

196/196 [=====] - 314s 2s/step - loss_gp:
2.0392 - discriminator_loss: -56.1984 - disc_loss: -35.8060 - rewards:
0.0952 - connectivity_g: 0.0951 - gen_loss: 10.7136 - critic_g_fake: -
2.6820 - critic_g_fake_loss: 2.7202 - time: 1.5575

Epoch 410/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0264 -
discriminator_loss: -55.9279 - disc_loss: -35.6636 - rewards: 0.0946 -
connectivity_g: 0.0946 - gen_loss: 10.9377 - critic_g_fake: -2.8743 -
critic_g_fake_loss: 2.9346 - time: 1.5623

Generating and plotting samples at epoch 409

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
543.40it/s]

196/196 [=====] - 314s 2s/step - loss_gp:
2.0264 - discriminator_loss: -55.9279 - disc_loss: -35.6636 - rewards:
0.0946 - connectivity_g: 0.0946 - gen_loss: 10.9377 - critic_g_fake: -
2.8743 - critic_g_fake_loss: 2.9346 - time: 1.5571

Epoch 411/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0453 -
discriminator_loss: -56.3961 - disc_loss: -35.9432 - rewards: 0.0997 -
connectivity_g: 0.0996 - gen_loss: 14.6252 - critic_g_fake: -2.8218 -
critic_g_fake_loss: 2.6722 - time: 1.5602

Generating and plotting samples at epoch 410

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
527.20it/s]

196/196 [=====] - 313s 2s/step - loss_gp:
2.0453 - discriminator_loss: -56.3961 - disc_loss: -35.9432 - rewards:
0.0997 - connectivity_g: 0.0996 - gen_loss: 14.6252 - critic_g_fake: -
2.8218 - critic_g_fake_loss: 2.6722 - time: 1.5551

Epoch 412/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0408 -

discriminator_loss: -56.3049 - disc_loss: -35.8965 - rewards: 0.1022 -
connectivity_g: 0.1022 - gen_loss: 14.8104 - critic_g_fake: -2.9679 -
critic_g_fake_loss: 2.9379 - time: 1.5635

Generating and plotting samples at epoch 411

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
516.04it/s]

196/196 [=====] - 314s 2s/step - loss_gp:
2.0408 - discriminator_loss: -56.3049 - disc_loss: -35.8965 - rewards:
0.1022 - connectivity_g: 0.1022 - gen_loss: 14.8104 - critic_g_fake: -
2.9679 - critic_g_fake_loss: 2.9379 - time: 1.5584

Epoch 413/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0426 -
discriminator_loss: -56.4698 - disc_loss: -36.0441 - rewards: 0.1081 -
connectivity_g: 0.1081 - gen_loss: 13.6226 - critic_g_fake: -2.2946 -
critic_g_fake_loss: 2.3036 - time: 1.5562

Generating and plotting samples at epoch 412

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
528.13it/s]

=== connected_validity plateaued. Increasing value_for_ones to 20.0

===

196/196 [=====] - 313s 2s/step - loss_gp:
2.0426 - discriminator_loss: -56.4698 - disc_loss: -36.0441 - rewards:
0.1081 - connectivity_g: 0.1081 - gen_loss: 13.6226 - critic_g_fake: -
2.2946 - critic_g_fake_loss: 2.3036 - time: 1.5510

Epoch 414/500

196/196 [=====] - ETA: 0s - loss_gp: 2.0622 -
discriminator_loss: -56.8212 - disc_loss: -36.1990 - rewards: 0.1213 -
connectivity_g: 0.1213 - gen_loss: 34.7029 - critic_g_fake: -2.9182 -
critic_g_fake_loss: 2.9792 - time: 1.5681

Generating and plotting samples at epoch 413

4/4 [=====] - 0s 8ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
540.84it/s]

196/196 [=====] - 315s 2s/step - loss_gp:
2.0622 - discriminator_loss: -56.8212 - disc_loss: -36.1990 - rewards:
0.1213 - connectivity_g: 0.1213 - gen_loss: 34.7029 - critic_g_fake: -
2.9182 - critic_g_fake_loss: 2.9792 - time: 1.5630

Epoch 415/500

196/196 [=====] - ETA: 0s - loss_gp: 2.1008 -
discriminator_loss: -57.9286 - disc_loss: -36.9209 - rewards: 0.1504 -
connectivity_g: 0.1504 - gen_loss: 50.3625 - critic_g_fake: -3.9616 -
critic_g_fake_loss: 3.9467 - time: 1.5910

Generating and plotting samples at epoch 414

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
549.73it/s]

196/196 [=====] - 319s 2s/step - loss_gp:
2.1008 - discriminator_loss: -57.9286 - disc_loss: -36.9209 - rewards:
0.1504 - connectivity_g: 0.1504 - gen_loss: 50.3625 - critic_g_fake: -
3.9616 - critic_g_fake_loss: 3.9467 - time: 1.5859

Epoch 416/500

196/196 [=====] - ETA: 0s - loss_gp: 2.1049 -
discriminator_loss: -57.9659 - disc_loss: -36.9165 - rewards: 0.1561 -
connectivity_g: 0.1562 - gen_loss: 53.6569 - critic_g_fake: -3.2006 -
critic_g_fake_loss: 3.2869 - time: 1.5942

Generating and plotting samples at epoch 415

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
544.63it/s]

196/196 [=====] - 320s 2s/step - loss_gp:
2.1049 - discriminator_loss: -57.9659 - disc_loss: -36.9165 - rewards:
0.1561 - connectivity_g: 0.1562 - gen_loss: 53.6569 - critic_g_fake: -
3.2006 - critic_g_fake_loss: 3.2869 - time: 1.5890

Epoch 417/500

196/196 [=====] - ETA: 0s - loss_gp: 2.1172 -
discriminator_loss: -58.1311 - disc_loss: -36.9591 - rewards: 0.1421 -
connectivity_g: 0.1423 - gen_loss: 36.7881 - critic_g_fake: -2.3225 -
critic_g_fake_loss: 2.4210 - time: 1.5745

Generating and plotting samples at epoch 416

4/4 [=====] - 0s 8ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
535.46it/s]

196/196 [=====] - 316s 2s/step - loss_gp:
2.1172 - discriminator_loss: -58.1311 - disc_loss: -36.9591 - rewards:
0.1421 - connectivity_g: 0.1423 - gen_loss: 36.7881 - critic_g_fake: -
2.3225 - critic_g_fake_loss: 2.4210 - time: 1.5694

Epoch 418/500

196/196 [=====] - ETA: 0s - loss_gp: 2.1251 -
discriminator_loss: -58.6902 - disc_loss: -37.4390 - rewards: 0.1333 -
connectivity_g: 0.1333 - gen_loss: 34.0901 - critic_g_fake: -2.8135 -
critic_g_fake_loss: 2.8313 - time: 1.5767

Generating and plotting samples at epoch 417

4/4 [=====] - 0s 9ms/step

Generating molecules: 100%|██████████| 100/100 [00:00<00:00,
536.58it/s]

```
196/196 [=====] - 317s 2s/step - loss_gp:
2.1251 - discriminator_loss: -58.6902 - disc_loss: -37.4390 - rewards:
0.1333 - connectivity_g: 0.1333 - gen_loss: 34.0901 - critic_g_fake: -
2.8135 - critic_g_fake_loss: 2.8313 - time: 1.5715
```

Epoch 419/500

```
117/196 [=====>.....] - ETA: 2:05 - loss_gp: 2.1226
- discriminator_loss: -58.4098 - disc_loss: -37.1843 - rewards: 0.1266
- connectivity_g: 0.1266 - gen_loss: 28.5726 - critic_g_fake: -2.3854
- critic_g_fake_loss: 2.3854 - time: 1.5830
```

```
-----
-----
KeyboardInterrupt                                Traceback (most recent call
last)
```

Cell In[20], line 21

```
18     steps_per_epoch += 1
20 # Train the model
---> 21 wgan.fit(
22     data_gen,
23     initial_epoch=starting_epoch,
24     epochs= 500,
25     steps_per_epoch=steps_per_epoch,
26     callbacks=[
27         checkpoint_callback,
28         plot_samples_callback,
29         gan_logger,
30         ResetMetricsCallback()
31     ],
32 )
33 # Save the trained model
34 wgan.save_model()
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/keras/
utils/traceback_utils.py:65, in
filter_traceback.<locals>.error_handler(*args, **kwargs)
63 filtered_tb = None
64 try:
---> 65     return fn(*args, **kwargs)
66 except Exception as e:
67     filtered_tb = _process_traceback_frames(e.__traceback__)
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/keras/
engine/training.py:1685, in Model.fit(self, x, y, batch_size, epochs,
verbose, callbacks, validation_split, validation_data, shuffle,
class_weight, sample_weight, initial_epoch, steps_per_epoch,
validation_steps, validation_batch_size, validation_freq,
max_queue_size, workers, use_multiprocessing)
1677 with tf.profiler.experimental.Trace(
```

```

1678     "train",
1679     epoch_num=epoch,
1680     ...)
1681     _r=1,
1682 ):
1683     callbacks.on_train_batch_begin(step)
-> 1684     tmp_logs = self.train_function(iterator)
1685     if data_handler.should_sync:
1686         context.async_wait()

```

File

~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/keras/engine/training.py:1284, in

Model.make_train_function.<locals>.train_function(iterator)

```

1282 def train_function(iterator):
1283     """Runs a training execution with a single step."""
-> 1284     return step_function(self, iterator)

```

File

~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/keras/engine/training.py:1268, in

Model.make_train_function.<locals>.step_function(model, iterator)

```

1264     run_step = tf.function(
1265         run_step, jit_compile=True, reduce_retracing=True
1266     )
1267     data = next(iterator)
-> 1268     outputs = model.distribute_strategy.run(run_step,
args=(data,))
1269     outputs = reduce_per_replica(
1270         outputs,
1271         self.distribute_strategy,
1272         reduction=self.distribute_reduction_method,
1273     )
1274     return outputs

```

File

~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow/python/distribute/distribute_lib.py:1316, in

StrategyBase.run(**failed resolving arguments**)

```

1311 with self.scope():
1312     # tf.distribute supports Eager functions, so AutoGraph
should not be
1313     # applied when the caller is also in Eager mode.
1314     fn = autograph.tf_convert(
1315         fn, autograph_ctx.control_status_ctx(),
convert_by_default=False)
-> 1316     return self._extended.call_for_each_replica(fn, args=args,
kwargs=kwargs)

```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow/
/python/distribute/distribute_lib.py:2895, in
StrategyExtendedV1.call_for_each_replica(self, fn, args, kwargs)
    2893     kwargs = {}
    2894     with self._container_strategy().scope():
-> 2895     return self._call_for_each_replica(fn, args, kwargs)
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow/
/python/distribute/distribute_lib.py:3696, in
_DefaultDistributionExtended._call_for_each_replica(self, fn, args,
kwargs)
    3694 def _call_for_each_replica(self, fn, args, kwargs):
    3695     with ReplicaContext(self._container_strategy(),
replica_id_in_sync_group=0):
-> 3696     return fn(*args, **kwargs)
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow/
/python/autograph/impl/api.py:595, in
call_with_unspecified_conversion_status.<locals>.wrapper(*args,
**kwargs)
    593 def wrapper(*args, **kwargs):
    594     with
ag_ctx.ControlStatusCtx(status=ag_ctx.Status.UNSPECIFIED):
--> 595     return func(*args, **kwargs)
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/keras/
engine/training.py:1249, in
Model.make_train_function.<locals>.step_function.<locals>.run_step(dat
a)
    1248 def run_step(data):
-> 1249     outputs = model.train_step(data)
    1250     # Ensure counter is updated only if `train_step` succeeds.
    1251     with
tf.control_dependencies(_minimum_control_deps(outputs)):
```

```
Cell In[10], line 115, in GraphWGAN.train_step(self, inputs)
    112     discriminator_loss = tf.reduce_mean(critic_fake) -
tf.reduce_mean(critic_real)
    113     disc_loss = discriminator_loss + loss_gp * self.gp_weight
--> 115     gradients_of_discriminator = disc_tape.gradient(disc_loss,
self.discriminator.trainable_variables)
    116
self.optimizer_discriminator.apply_gradients(zip(gradients_of_discrimi
nator, self.discriminator.trainable_variables))
    118 # Update the metric values
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow
/python/eager/backprop.py:1063, in GradientTape.gradient(self, target,
sources, output_gradients, unconnected_gradients)
    1057     output_gradients = (
    1058
composite_tensor_gradient.get_flat_tensors_for_gradients(
    1059         output_gradients))
    1060     output_gradients = [None if x is None else
ops.convert_to_tensor(x)
    1061         for x in output_gradients]
-> 1063 flat_grad = imperative_grad.imperative_grad(
    1064     self._tape,
    1065     flat_targets,
    1066     flat_sources,
    1067     output_gradients=output_gradients,
    1068     sources_raw=flat_sources_raw,
    1069     unconnected_gradients=unconnected_gradients)
    1071 if not self._persistent:
    1072     # Keep track of watched variables before setting tape to
None
    1073     self._watched_variables = self._tape.watched_variables()
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow
/python/eager/imperative_grad.py:67, in imperative_grad(tape, target,
sources, output_gradients, sources_raw, unconnected_gradients)
    63 except ValueError:
    64     raise ValueError(
    65         "Unknown value for unconnected_gradients: %r" %
unconnected_gradients)
---> 67 return pywrap_tfe.TFE_Py_TapeGradient(
    68     tape._tape, # pylint: disable=protected-access
    69     target,
    70     sources,
    71     output_gradients,
    72     sources_raw,
    73     compat.as_str(unconnected_gradients.value))
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow
/python/eager/backprop.py:146, in _gradient_function(op_name,
attr_tuple, num_inputs, inputs, outputs, out_grads,
skip_input_indices, forward_pass_name_scope)
    144     gradient_name_scope += forward_pass_name_scope + "/"
    145     with ops.name_scope(gradient_name_scope):
--> 146         return grad_fn(mock_op, *out_grads)
    147 else:
    148     return grad_fn(mock_op, *out_grads)
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow
/python/ops/math_grad.py:256, in _MeanGrad(op, grad)
    254     output_size = np.prod(output_shape)
    255     factor = input_size // max(output_size, 1)
--> 256     factor = constant_op.constant(factor, dtype=sum_grad.dtype)
    257 else:
    258     input_shape = array_ops.shape(op.inputs[0])
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow
/python/framework/constant_op.py:268, in constant(value, dtype, shape,
name)
    171 @tf_export("constant", v1=[])
    172 def constant(value, dtype=None, shape=None, name="Const"):
    173     """Creates a constant tensor from a tensor-like object.
    174
    175     Note: All eager `tf.Tensor` values are immutable (in
contrast to
    (...)
    266     ValueError: if called on a symbolic tensor.
    267     """
--> 268     return _constant_impl(value, dtype, shape, name,
verify_shape=False,
    269                             allow_broadcast=True)
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow
/python/framework/constant_op.py:280, in _constant_impl(value, dtype,
shape, name, verify_shape, allow_broadcast)
    278     with trace.Trace("tf.constant"):
    279         return _constant_eager_impl(ctx, value, dtype, shape,
verify_shape)
--> 280     return _constant_eager_impl(ctx, value, dtype, shape,
verify_shape)
    282 g = ops.get_default_graph()
    283 tensor_value = attr_value_pb2.AttrValue()
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow
/python/framework/constant_op.py:305, in _constant_eager_impl(ctx,
value, dtype, shape, verify_shape)
    303 def _constant_eager_impl(ctx, value, dtype, shape,
verify_shape):
    304     """Creates a constant on the current device."""
--> 305     t = convert_to_eager_tensor(value, ctx, dtype)
    306     if shape is None:
    307         return t
```

File

```
~/miniconda3/envs/my-rdkit-env/lib/python3.10/site-packages/tensorflow
```

```
/python/framework/constant_op.py:103, in  
convert_to_eager_tensor(value, ctx, dtype)  
    101     dtype = dtypes.as_dtype(dtype).as_datatype_enum  
    102     ctx.ensure_initialized()  
--> 103     return ops.EagerTensor(value, ctx.device_name, dtype)
```

KeyboardInterrupt: